

SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)
Department of Computer Science and Engineering

ASSESSMENT TOOL 3 – SHORT ANSWER TEST

Course Code:	CSA0305	Course Title:	Data Structures
CO Assessed:	CO5 – Manipulation (BL3, BL4)	Assessment:	Assessment Tool 3 – Short Answer Test
Weightage:	30% of CIA	Total Marks:	100
CO2 Statement:	Construct MST using shortest path algorithms and traverse the graph to model and analyse real-world problem.		
Student Name:	S. Dharshini	Reg. No.:	192511260
Section / Batch:	2025- CSE	Date:	04/09/2026

Code of Conduct

I S. Dharshini (192511260) (Name / Reg No)

certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: S. Dharshini

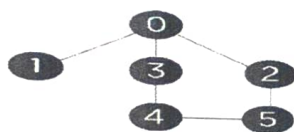
Instructions

- This test contains 10 short answer test questions. Total: 100 Marks.
- When a student answer using an Analytical problem-solving, in evaluation the answer should be with following five requirements: Understanding of problem, select appropriate data structure, Design the solution, analyze, Clarity of representation.
- No negative marking. Unanswered questions carry zero marks.
- No DTP printouts or electronic devices permitted. They should provide written materials.

Section / Topic	Focus	Q Nos	Marks
Minimum Spanning Tree	Kruskal's Algorithm	1	20
Graph Sorting	Topological Sort	2	20
Shortest Path Algorithm	MST & Dijkstra's Algorithm	3	20
Shortest Path Algorithm	Dijkstra's Algorithm	4	20
Minimum Spanning Tree	Prim's or Kruskal's Algorithm	5	20
GRAND TOTAL:			100 Marks

Annexure A - Short Answer Test

- Perform the following operations using stack. Assume the size of the stack is 5 and have a value of 22, 55, 33, 66, 88 in the stack from 0 position to size-1. Now perform the following operations: 1) Invert the elements in the stack, 2) Pop (), 3) Pop (), 3) Pop Construct AVL tree for the following elements. Mention the
- Given a collection of elements 10,20,30,40,50,60,70,80. The Key element to be search 70. Compare how searching is performed in Linear Search and Binary Search. Identify the efficient search method. Justify.
- What is the postfix and prefix forms of the following expression? $A+B*(C-D)/(P-R)$
- Construct AVL tree for the following elements. Mention the Rotations in each step of insertion. 1,2,3,4,5,6,7,8,9,10
- Construct a heap by inserting 10, 15, 12, 23, 21, 1, 6, 43, 34, 56, 78, 23, 45 into a binary heap.
- Construct a B-Tree of Order 3 by inserting numbers 34,26,45,12,87,56,78,22,23,24,65,85,95.
- Perform the merge sort for the following inputs. Also, show the result for each step of iteration. 40,20,70,14,60,61,97,30.
- How the following data set can be sorted by quick sort. Show each step-in sorting. 78,23,31,88,43,55,67,55.
- Perform Open Hashing & Linear Probing for the given set of 0,1,81, 4,64,25,16,36, 9 and 49.
- Consider the following graph & construct BFS.



Graph with six nodes

Rubrics for Calculation

Criteria	Weightage	Level 4 - Exemplary	Level 3 - Proficient	Level 2 - Developing	Level 1 - Beginning
Conceptual Understanding	30	Demonstrates complete and accurate understanding of concepts	Good understanding with minor gaps	Basic understanding	Limited understanding
Accuracy of Answer	25	Answers are completely correct and relevant	Mostly correct with minor errors	Partially correct	Mostly incorrect
Problem-Solving & Reasoning	20	Provides logical reasoning and appropriate steps	Good reasoning with minor gaps	Basic reasoning	Lacks logical reasoning
Clarity & Relevance	15	Answers are precise, clear, concise, and directly address the question	Mostly clear and relevant	Somewhat unclear or incomplete	Irrelevant or unclear answers
Time Management & Completion	10	Completes all questions accurately within the allotted time	Completes most questions on time	Requires additional time	Unable to complete the test

Faculty In-charge	Course Coordinator	Program Director
Signature: 	Signature: _____	Signature: _____
Date: _____	Date: _____	Date: _____

STACK OPERATIONS

Given Stack Size = 5.

Position	0	1	2	3	4
Element	22	55	33	66	88

Assuming position 4 is the TOP.

TOP → 88
66
33
55
22

Operation 1: Invert the stack
After inversion

TOP → 22
55
33
66
88

Operation 2: POP()

Remove 22
TOP → 55
33
66
88

Operation 3: POP()

~~Remove 55~~
TOP → 33
66
88

Operation 4: POP()

Remove 33
TOP → 66
88

Final stack = 66, 88

2. Linear Search Vs Binary Search

Given:-

10, 20, 30, 40, 50, 60, 70, 80

Search key = 70

Linear Search

10 $\rightarrow$ 20 $\rightarrow$ 30 $\rightarrow$ 40 $\rightarrow$ 50 $\rightarrow$ 60 $\rightarrow$ 70

Comparison = 7

Binary Search

The data is sorted, so binary search can be used.

Initial: 10 20 30 40 50 60 70 80

Middle = 40

Since: $70 > 40$

Search Right half:- 50 60 70 80

Middle = 60

Since: $70 > 60$

Search: 70 80

Middle = 70

3.

Prefix and Postfix

Expression: $A + B * (C + D) / (P - R)$

Step 1: $(C + D)$

Postfix: $CD +$

Prefix: $+(CD)$

Step 2: $B * (C + D)$

Postfix: $BCD + *$

Prefix: $*(BCD +)$

Step 3:- $(P - R)$

Postfix: $PR -$

Prefix: $-(PR)$

Infix : $A + B * (C - D) / (P - R)$
 Postfix : $ABCD - * PR - / +$
 Prefix : $+ A / * B - CD - PR$

4.

AVL Tree : 1, 2, 3, 4, 5, 6, 7, 8, 9, 10

Insert 1

No rotation

Insert 2

1 \ 2

No rotation

Insert 3

Before Balancing:-

1 \ 2 \ 3

This is RR imbalance at 1
 Perform Left Rotation at 1

2
 1 \ 3

Insert 4

2
 1 \ 3
 4

No rotation

Insert 5

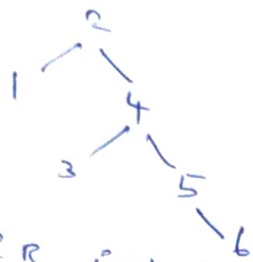
Before Balancing

2
 1 \ 3
 4 \ 5

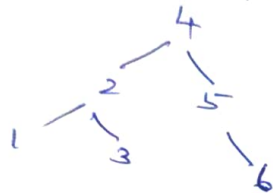
RR imbalance at 3

Perform left rotation at 3

Insert 6

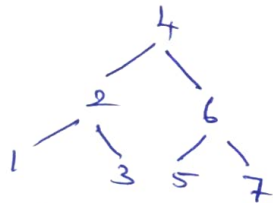


RR imbalance at 2
perform Left Rotation at 2

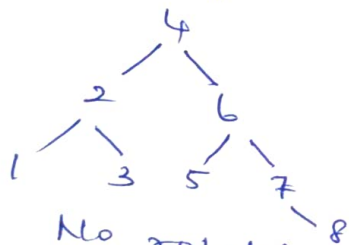


Insert 7

RR imbalance at 5
Left Rotation at 5:



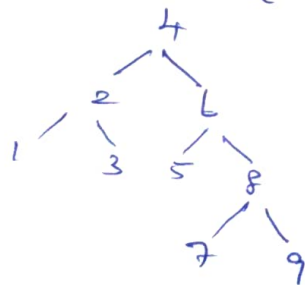
Insert 8



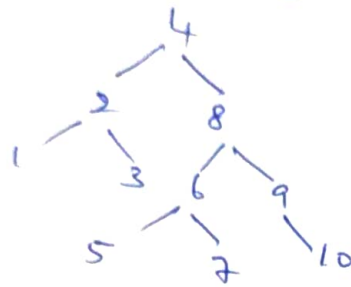
No rotation

Insert 9

RR imbalance at 7
Left Rotation at 7:



Insert 10
 RR imbalance at 6
 Left rotation at 6:

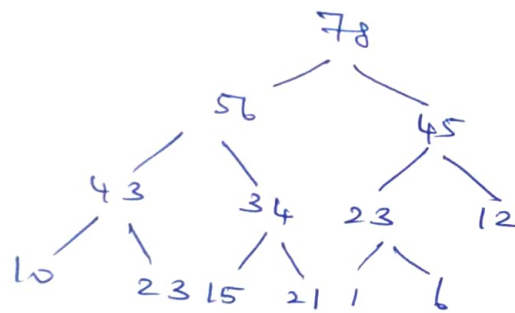


5.

Binary Heap.

Elements: 10, 15, 12, 23, 21, 1, 6, 43, 34, 56, 78, 23, 45

Assuming a Max Heap



Array Representation: [78, 56, 45, 43, 34, 23, 12, 10, 23, 15, 21, 1, 6]

Heap Property

Every parent is greater than or equal to its children:-

$$78 > 56, 45$$

$$56 > 43, 34$$

$$45 > 23, 12$$

$$43 > 10, 23$$

$$34 > 15, 21$$

$$23 > 1, 6$$

∴, it is a valid Max Heap

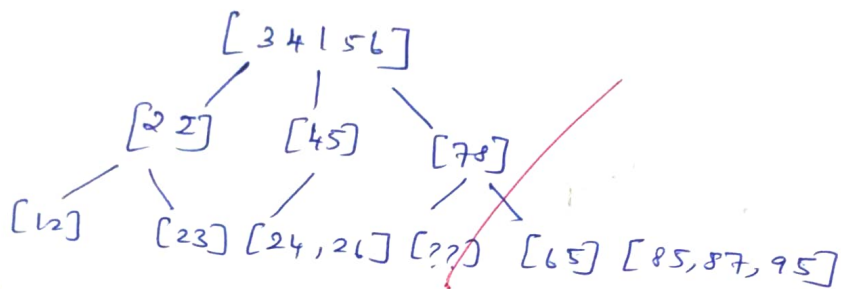
6.

B-Tree of order 3

Input:

34, 26, 45, 12, 87, 56, 78, 22, 23, 24, 65, 85, 95

Order-3 B-Tree each node have at most 3 children and 2 keys.



However, this depends on the exact B-tree convention used by your faculty - particularly how they define "order 3" and split/promote nodes.

7.

Merge Sort

Input:

40, 20, 70, 14, 60, 61, 97, 30

Iteration 1: Divide

[40, 20, 70, 14] [60, 61, 97, 30]

Divide again:

[40, 20] [70, 14] [60, 61] [97, 30]

Divide:

[40] [20] [70] [14] [60] [61] [97] [30]

Iteration 2: Merge

[40] + [20] → [20, 40]

[70] + [14] → [14, 70]

[60] + [61] → [60, 61]

[97] + [30] → [30, 97]

Now:

[20, 40] [14, 70] [60, 61] [30, 97]

arks

20

20

20

20

20

20

Iteration 5

$$[20, 40] + [14, 70]$$

Compare :-

14 $\rightarrow$ 20 $\rightarrow$ 40 $\rightarrow$ 70
Result: 14

Result: [14, 20, 40, 70]
other nodes

Other mol:-

$$[60, 61] + [30, 97]$$

Result:

[30, 60, 61, 92]

Final merge = [14, 20, 40, 70] + [30, 60, 61, 92]

8.

Quick Sort

78, 23, 31, 88, 43, 55, 67, 55

Pivot = 55

78, 23, 31, 88, 43, 55, 67, 55
 after partition: ↑
pivot

↑
Pivot

After partition:-

23, 31, 43, 55, 55, 78, 67, 88

Left part

23, 31, 43, ~~55~~

Pivot = ~~55~~

23, 31, 43, 155

Left part

23, 31, 43

Pivot = 43

23, 31 | 43

Pivot = 31

23131

23, 31, 43, 55

Right Part

78, 67, 88

Pivot = 88

78, 67, 88

Pivot = 67:

67, 78

∴ 67, 78, 88

23, 31, 43, 55, 55, 67, 78, 88

9.

Open Hashing & Linear Probing

Given: -

0, 1, 81, 4, 64, 25, 16, 36, 9, 49

$h(k) = k \bmod 10$

Hash Values

key	$k \bmod 10$
0	0
1	1
81	1
4	4
64	4
25	5
16	6
36	6
9	9
49	9

Insert 0

0 → Index 0

Insert 1

1 $\rightarrow$ index 1

Insert 81

Hash = 1, but index 1 occupied

Try index 2:-

81 $\rightarrow$ index 2

Insert 4

4 $\rightarrow$ index 4

Insert 64

Hash = 4 occupied

64 $\rightarrow$ index 5

Insert 25

Hash = 5 occupied

Try 6:-

25 $\rightarrow$ index 6

Insert 16

Hash = 6 occupied

16 $\rightarrow$ index 7

Insert 36

Hash = 6 occupied

7 occupied $\rightarrow$ 8 available,-

36 $\rightarrow$ index 8

Insert 9

9 $\rightarrow$ index 9

Insert 49

Hash = 9 occupied.

0 occupied

1 occupied

2 occupied

10.

BFS

1. Starting vertex
2. Queue operations
3. Visited vertices at every step
4. BFS traversal
5. BFS tree

Step 1: Start $\rightarrow$ A

Queue: A

Step 2 :-

Visit A

Queue: B, C

Step 3 :-

Visit B

Queue: C, D, E

BFS traversal

A $\rightarrow$ B $\rightarrow$ C $\rightarrow$ D $\rightarrow$ E

Wong